

Iteration 4 Final

Compact presentation deck

지침서 기준으로 다시 압축한 최종 발표본

핵심 원칙

본편은 30장 내외로 콤팩트하게, DP 상세 코드는 appendix 문서로 분리

지침 반영

양식#1/#2/#3, 빨간 변경표시, 교과서 DP 구조 비교, 코드 역할 표시

발표 흐름

기능 요약 → iteration 변화 → DP별 비교/코드 → 데모 순서

지침서 기준으로 본편을 콤팩트하게 재구성

중요한 요구사항만 본편에 두고 상세 코드는 보조문서로 넘긴다.

- 1 양식#1 전체 기능, 양식#2 확장 기능, 양식#3 Refactoring/DP 정리표 포함
- 2 직전 버전 대비 추가/변경사항은 빨간색으로 표시
- 3 DP마다 교과서 클래스 다이어그램과 우리 팀 구현을 비교
- 4 코드는 원 구성도 역할이 드러나는 핵심 skeleton만 제시

전체 기능 정리표

누적 기능 중 최종 발표에서 보여줄 핵심만 압축

영역	주요 기능	Iteration / DP
Search	직항/왕복/환승/다구간, 도시코드 검색	iter4 Composite / Factory
Booking	승객정보, 좌석, 결제, 확정	iter1-4 State
Payment	카드/KakaoPay/ApplePay/계좌/마일리지	iter4 Factory / Adapter
Ticket	e-Ticket 포맷, 버스 연계	iter4 Template / Observer
Refund	취소/환불, 수동 review	iter2 Strategy + iter4 UI

확장 기능 정리표

확장요소가 있는 기능만 정리

기본 기능	확장/변경	비고
공항 검색	도시코드가 여러 공항으로 확장	Composite
결제	결제수단별 processor 생성	Factory Method
마일리지	외부 Skypass API 감싸기	Adapter
좌석	좌석 add-on 요금 누적	Decorator
e-Ticket	다중 렌더 포맷	Template Method

Iteration별 Refactoring / Design Pattern

보고서와 발표자료에 반드시 들어가는 표

Iter	R/DP	적용처/내용요약
1st	DP#1 State	Reservation lifecycle
2nd	Refactoring#1 / DP#2 Strategy	RefundPolicy family
3rd	Refactoring#2 / DP#3 Observer	DomainEvent publisher/listener
4th	Refactoring#3	Swing → JavaFX
4th	DP#4-#9	Composite, Singleton, Factory, Template, Adapter, Decorator

직전 버전 대비 변경사항

빨간 항목이 iter4 최종 신규/변경

- 1 JavaFX 전환: FXML/CSS/Controller 구조
- 2 패턴 가이드 화면: 발표 중 코드 역할 바로 확인
- 3 예약 조회 편의성: 상세/e-Ticket, PNR 복사, 진행 재개
- 4 결제 편의성: 마일리지 차감 예정 금액, 버스 운행/좌석 선택

다이어그램은 본편에서 흐름만 잡고 상세는 보조문서로

지침서가 요구한 Use Case/Class/Sequence/State는 모두 연결된다.

- 1 Use Case: 검색, 예약, 결제, 조회, 취소/환불, 설정
- 2 Class Diagram: DP별 Reference vs Team 장표로 분리
- 3 Sequence: Payment Factory, Mileage Adapter, Seat Decorator, Bus Observer
- 4 State Diagram: Reservation state badge와 취소/환불 전이로 시연

시연 순서

iter3-final처럼 사용 흐름 중심으로 간다.

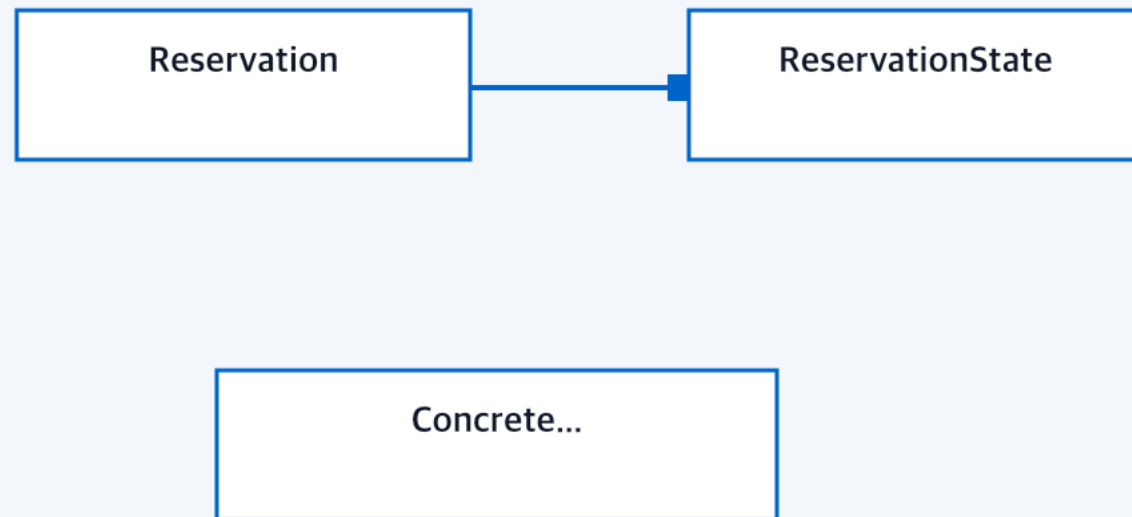
순서	화면	보여줄 DP
1	Search	Composite, Factory Method
2	Seat	Decorator
3	Payment	Factory Method, Adapter
4	Confirmation	Template Method, Observer
5	Lookup/Refund	State, Strategy
6	Settings/Guide	Singleton + 전체 DP map

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

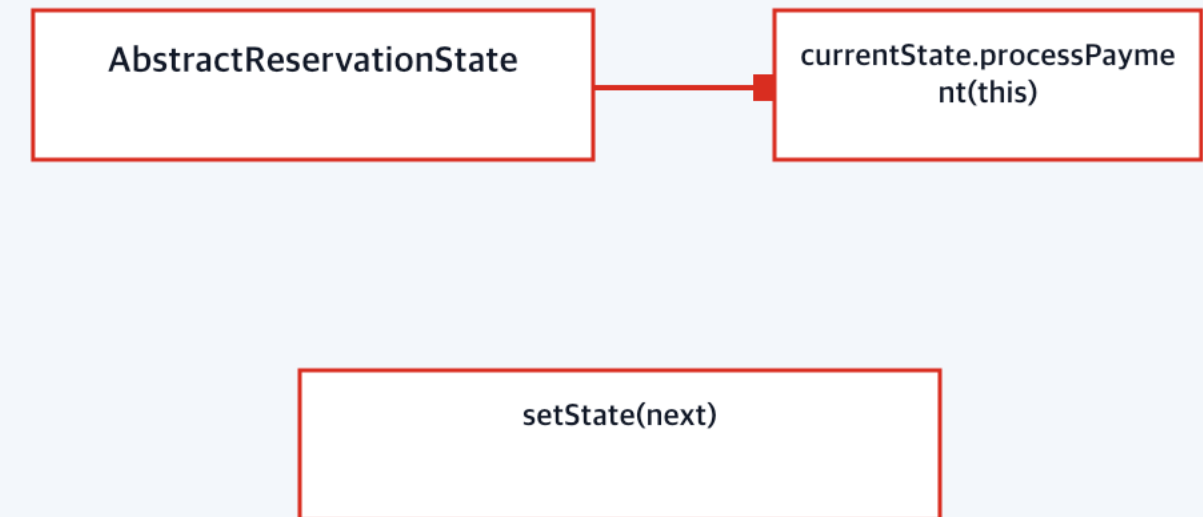
교과서 GoF

Reservation / ReservationState / Concrete classes



우리 구현

AbstractReservationState + 8 states



Ctrl+F: ReservationState / AbstractReservationState

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
class Reservation {  
    private ReservationState currentState;  
    void processPayment() {  
        currentState.processPayment(this);  
    }  
}  
abstract class AbstractReservationState  
    implements ReservationState {  
    void processPayment(Reservation r) {  
        throw new InvalidStateTransitionException(...);  
    }  
}
```

역할

Reservation = Context/Component/Creator
ReservationState = interface 또는 핵심 method

흐름

currentState.processPayment(this) → setState(next)

시연

ReservationState / AbstractReservationState

교과서 구조 vs 우리 팀 구조

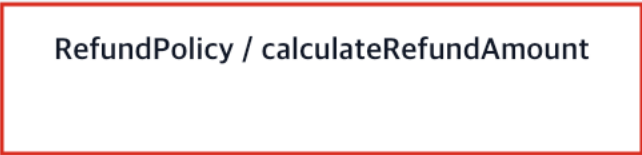
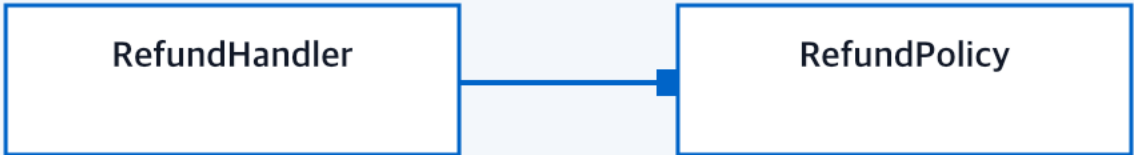
지침서 요구: 해당 디자인패턴 클래스 다이어그램과 팀 구현 비교

교과서 GoF

RefundHandler / RefundPolicy / Concrete classes

우리 구현

Full / Partial / NoRefundPolicy



Ctrl+F: RefundPolicy / calculateRefundAmount

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
interface RefundPolicy {  
    BigDecimal calculateRefundAmount(BigDecimal paid);  
}  
class RefundHandler {  
    RefundRequest evaluateRefund(...) {  
        RefundPolicy p = fareRule.checkRefundPolicy();  
        return request(p.calculateRefundAmount(paid));  
    }  
}
```

역할

RefundHandler = Context/Component/Creator
RefundPolicy = interface 또는 핵심 method

흐름

policy.calculateRefundAmount(paid)

시연

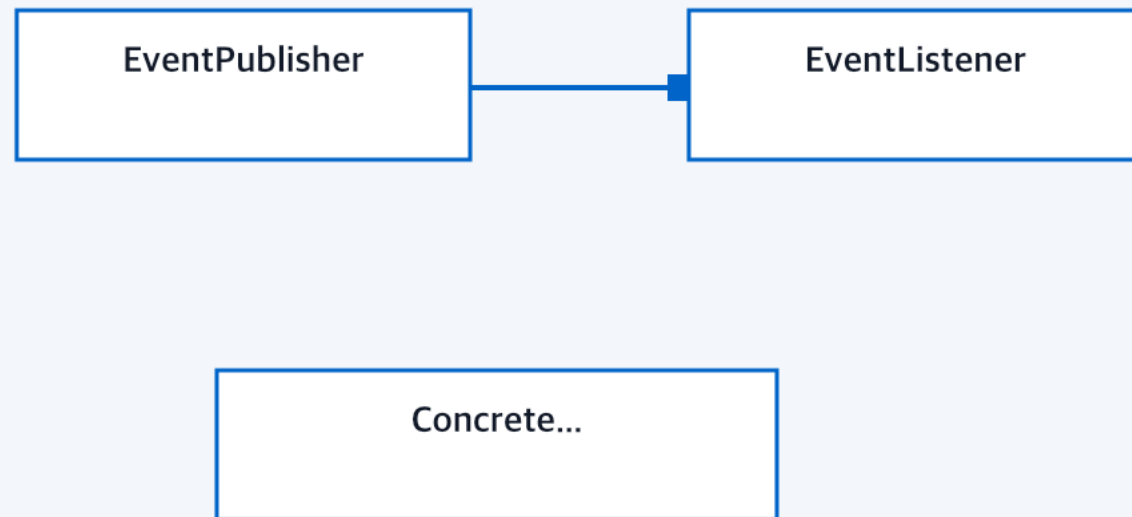
RefundPolicy / calculateRefundAmount

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스 다이어그램과 팀 구현 비교

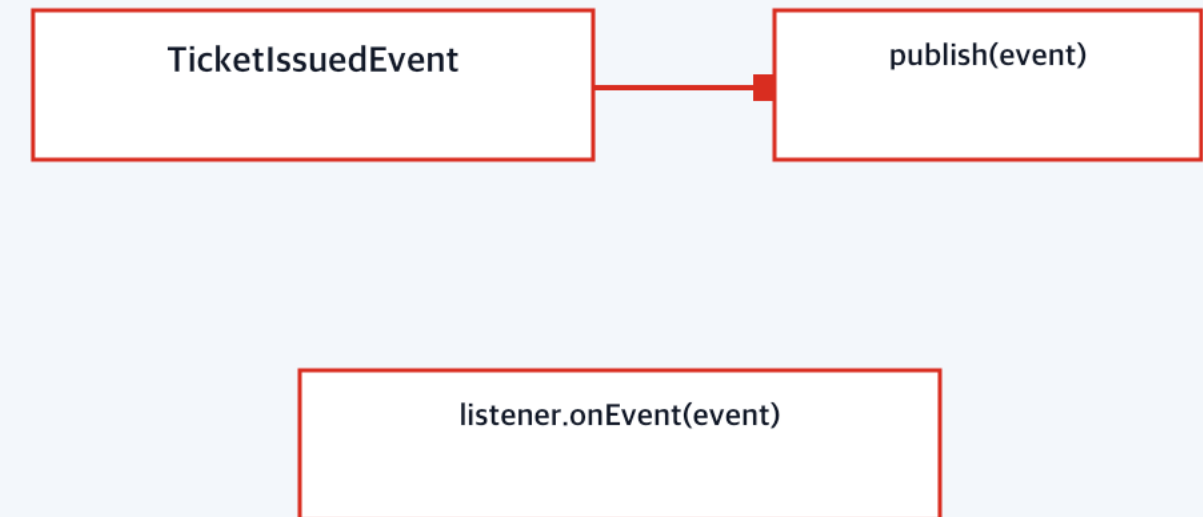
교과서 GoF

EventPublisher / EventListener / Concrete classes



우리 구현

TicketIssuedEvent + BusTicketPurchaseListener



Ctrl+F: EventPublisher / BusTicketPurchaseListener

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
class EventPublisher {  
    void publish(DomainEvent e) {  
        for (EventListener l : listeners) l.onEvent(e);  
    }  
}  
class BusTicketPurchaseListener implements EventListener {  
    void onEvent(DomainEvent e) { issuePremiumTicket(...); }  
}
```

역할

EventPublisher = Context/Component/Creator
EventListener = interface 또는 핵심 method

흐름

publish(event) → listener.onEvent(event)

시연

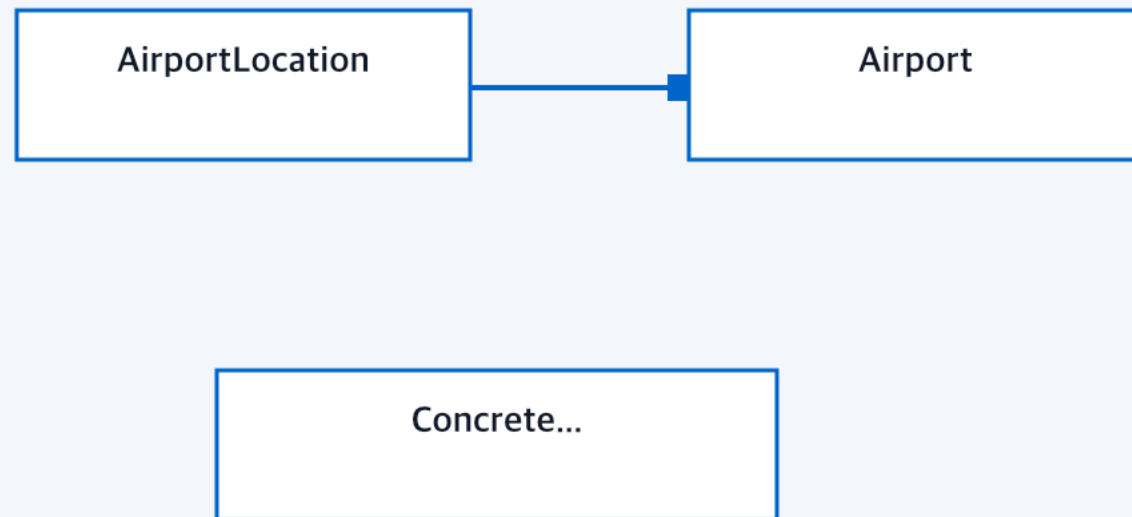
EventPublisher / BusTicketPurchaseListener

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스 다이어그램과 팀 구현 비교

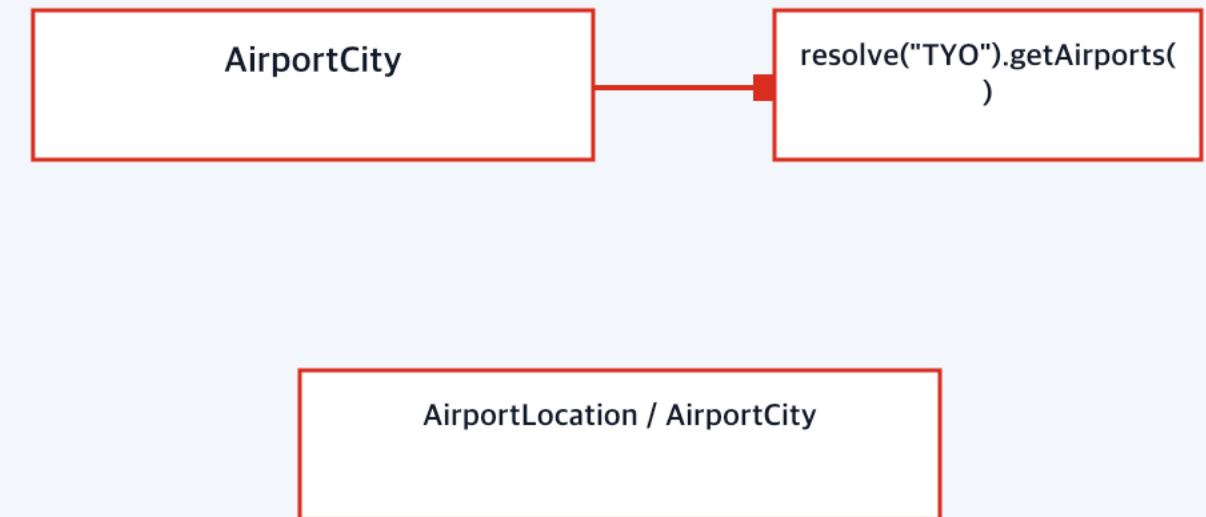
교과서 GoF

AirportLocation / Airport / Concrete classes



우리 구현

AirportCity



Ctrl+F: AirportLocation / AirportCity

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
interface AirportLocation {  
    List<Airport> getAirports();  
}  
class Airport implements AirportLocation {  
    List<Airport> getAirports() { return List.of(this); }  
}  
class AirportCity implements AirportLocation {  
    List<Airport> getAirports() { return airports; }  
}
```

역할

AirportLocation = Context/Component/Creator
Airport = interface 또는 핵심 method

흐름

resolve("TYO").getAirports()

시연

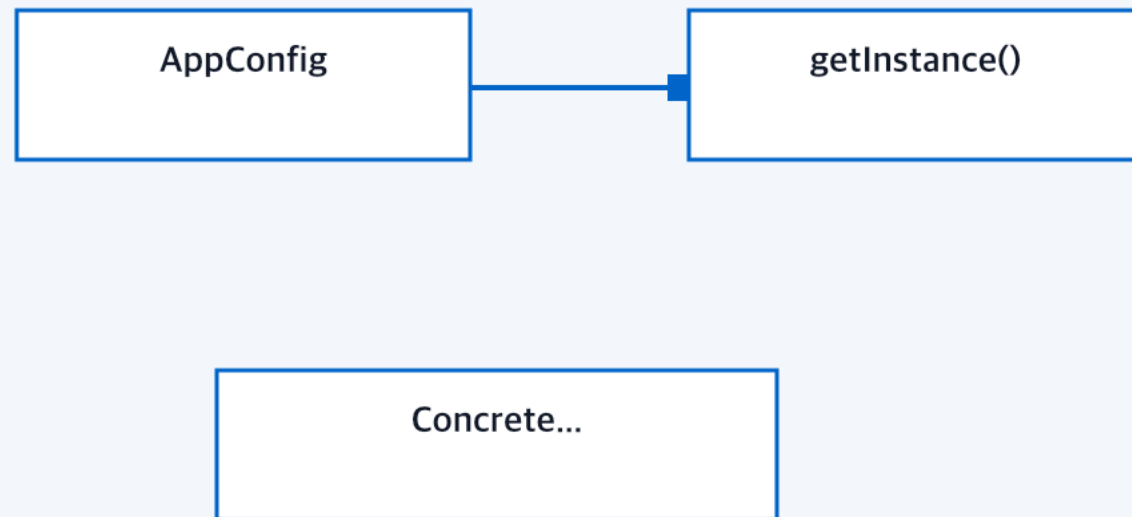
AirportLocation / AirportCity

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

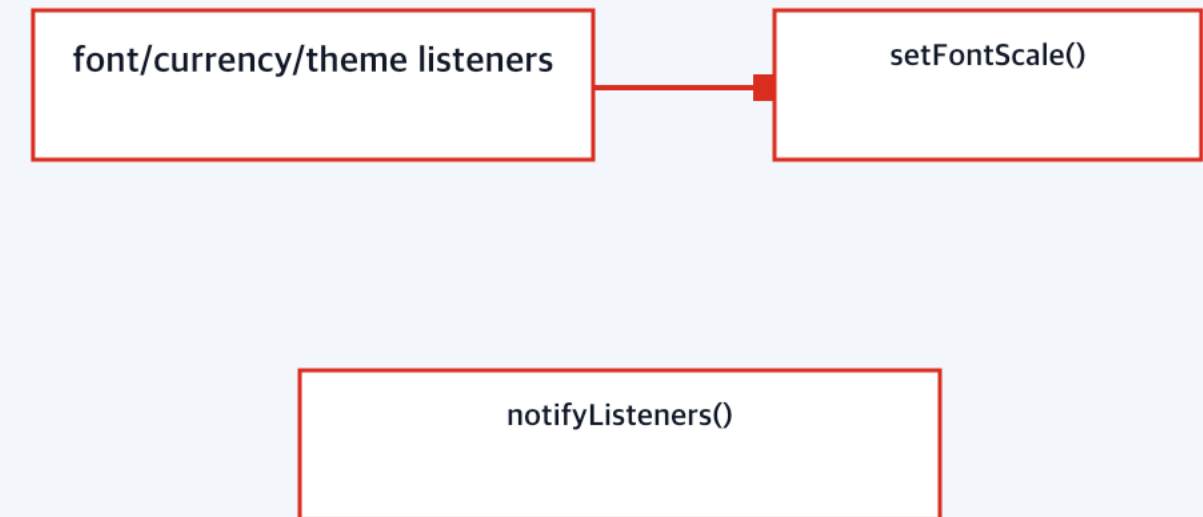
교과서 GoF

AppConfig / getInstance() / Concrete classes



우리 구현

font/currency/theme listeners



Ctrl+F: AppConfig / getInstance

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
public final class AppConfig {  
    private static volatile AppConfig instance;  
    private AppConfig() {}  
    static AppConfig getInstance() { ... }  
    void setFontScale(double v) {  
        fontScale = v; notifyListeners();  
    }  
}
```

역할

AppConfig = Context/Component/Creator
getInstance() = interface 또는 핵심 method

흐름

setFontScale() → notifyListeners()

시연

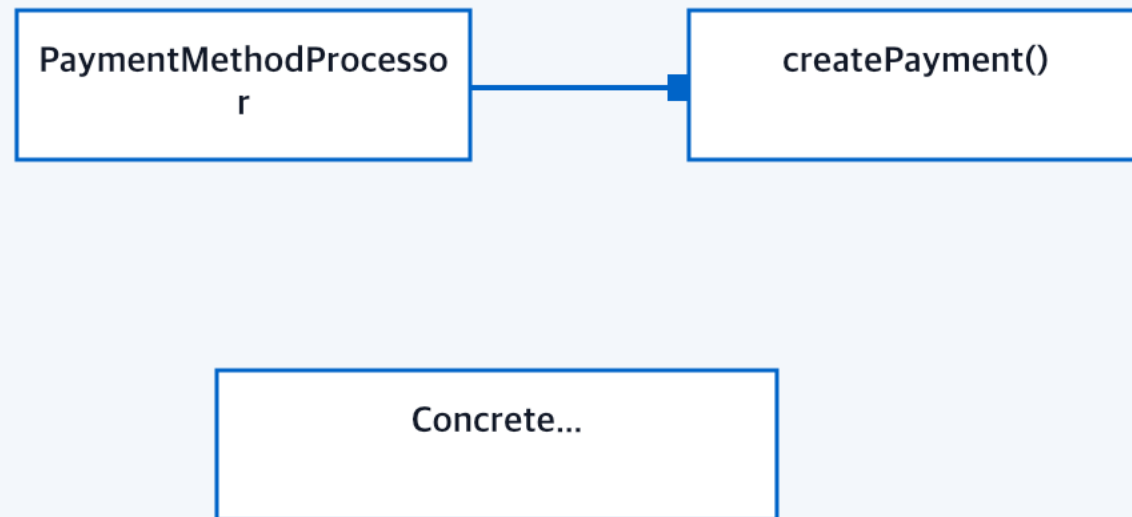
AppConfig / getInstance

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

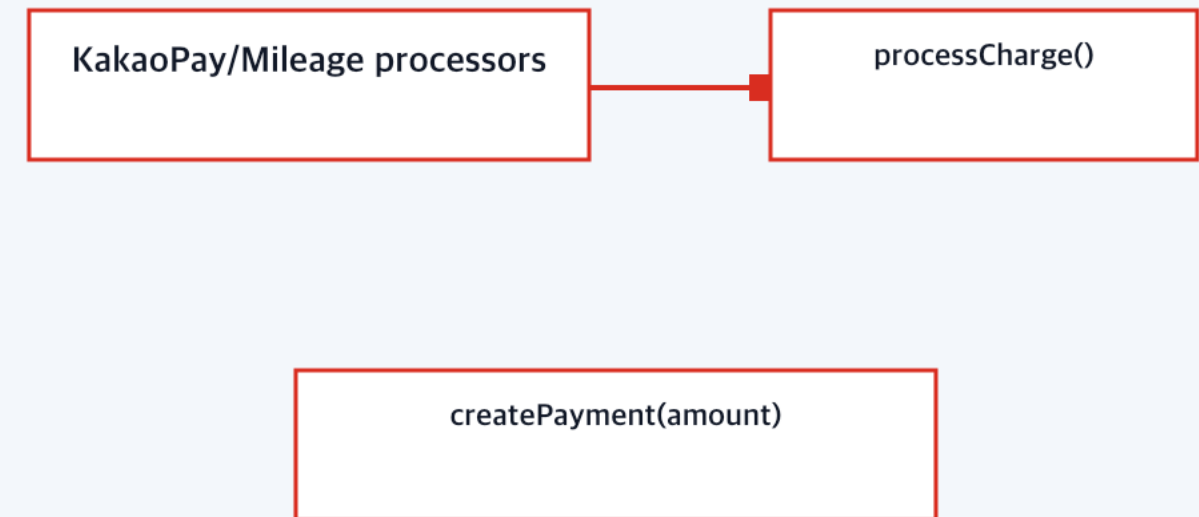
교과서 GoF

PaymentMethodProcessor / createPayment() / Concrete classes



우리 구현

KakaoPay/Mileage processors + ItineraryFactory



Ctrl+F: PaymentMethodProcessor / ItineraryFactory

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
abstract class PaymentMethodProcessor {  
    final Payment processCharge(long amount, String pnr) {  
        Payment p = createPayment(amount);  
        if (authorize(p)) p.pay();  
        return p;  
    }  
    protected abstract Payment createPayment(long amount);  
}
```

역할

PaymentMethodProcessor = Context/Component/Creator
createPayment() = interface 또는 핵심 method

흐름

processCharge() → createPayment(amount)

시연

PaymentMethodProcessor / ItineraryFactory

교과서 구조 vs 우리 팀 구조

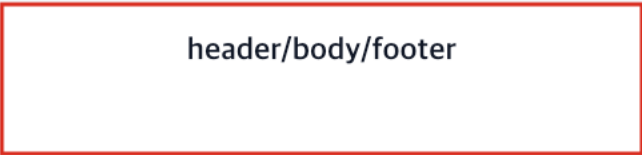
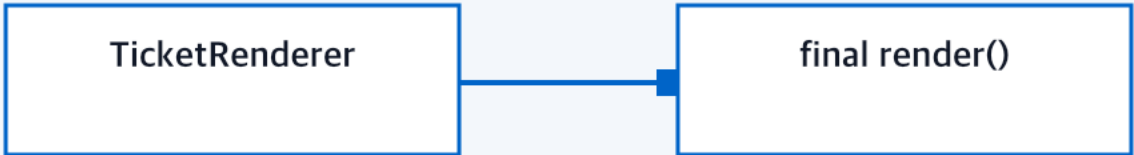
지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

교과서 GoF

TicketRenderer / final render() / Concrete classes

우리 구현

PlainText / HTML / BoardingPass



Ctrl+F: TicketRenderer / render

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
abstract class TicketRenderer {  
    public final String render(Reservation r, Ticket t) {  
        return header(r,t) + body(r,t) + footer(r,t);  
    }  
    protected abstract String header(...);  
    protected abstract String body(...);  
}
```

역할

TicketRenderer = Context/Component/Creator
final render() = interface 또는 핵심 method

흐름

render() → header/body/footer

시연

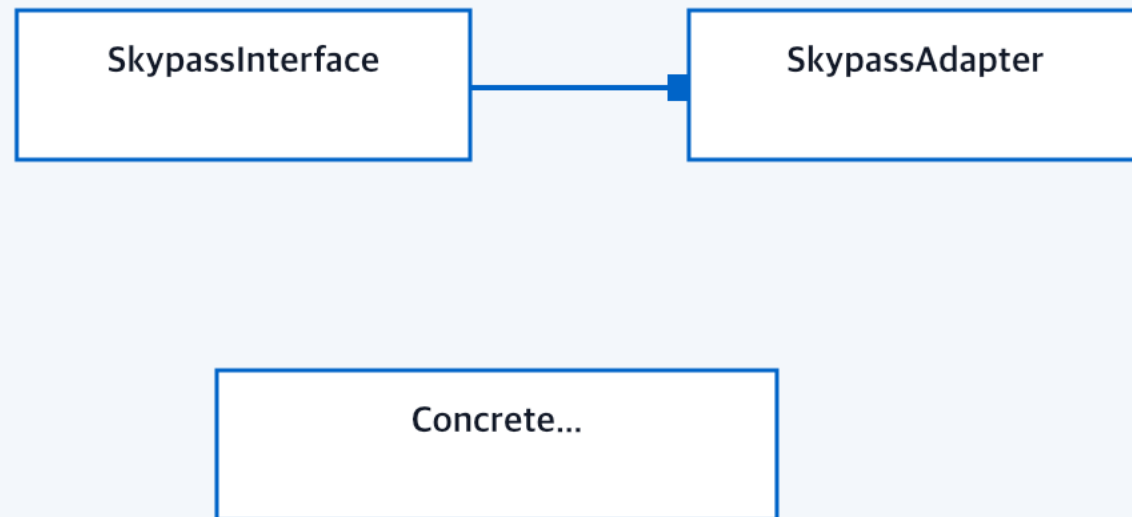
TicketRenderer / render

교과서 구조 vs 우리 팀 구조

지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

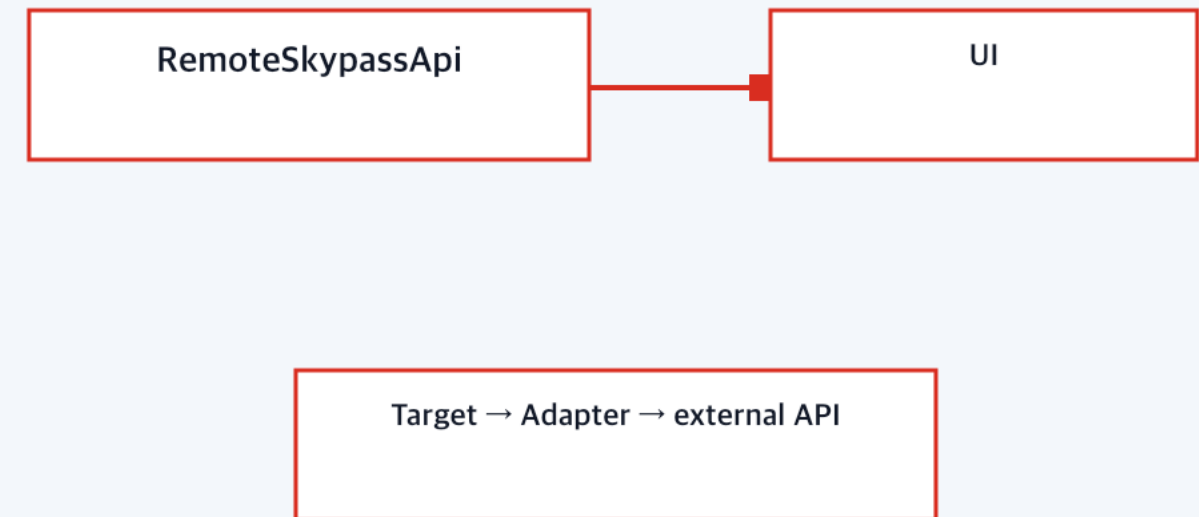
교과서 GoF

SkypassInterface / SkypassAdapter / Concrete classes



우리 구현

RemoteSkypassApi



Ctrl+F: SkypassAdapter / RemoteSkypassApi

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
class SkypassAdapter implements SkypassInterface {  
    private final RemoteSkypassApi remote;  
    int getMileageBalance(String no) {  
        Map<String, Object> res = remote.getMileage(no);  
        return (Integer) res.get("balance");  
    }  
}
```

역할

SkypassInterface = Context/Component/Creator
SkypassAdapter = interface 또는 핵심 method

흐름

UI → Target → Adapter → external API

시연

SkypassAdapter / RemoteSkypassApi

교과서 구조 vs 우리 팀 구조

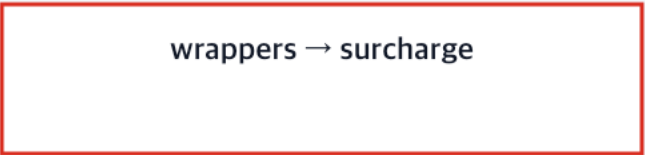
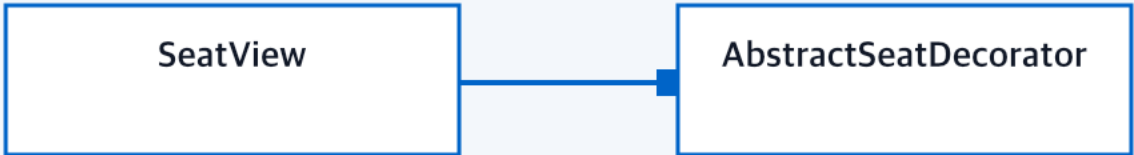
지침서 요구: 해당 디자인패턴 클래스다이어그램과 팀 구현 비교

교과서 GoF

SeatView / AbstractSeatDecorator / Concrete classes

우리 구현

Window/Aisle/Legroom/Lounge



Ctrl+F: SeatView / AbstractSeatDecorator

코드에서 원 구성도 역할 표시

코드 전체가 아니라 역할이 드러나는 부분만 보여준다

```
SeatView view = new SeatViewAdapter(seat);  
if (isWindow(seat)) view = new WindowSeatDecorator(view);  
if (legroom) view = new ExtraLegroomDecorator(view);  
if (lounge) view = new LoungeAccessDecorator(view);  
ctx.setSeatSurcharge(view.getSurcharge().longValue());
```

역할

SeatView = Context/Component/Creator
AbstractSeatDecorator = interface 또는 핵심 method

흐름

base view → wrappers → surcharge

시연

SeatView / AbstractSeatDecorator

검증 및 수정 이력

구현 완성도와 신뢰성 항목 대응

문제	위험	수정
double refund	환불 중복	cancel path와 review path 분리
fare hardcoding	다구간 금액 오류	segment count 기반 계산
lookup flow	예약 후 돌아가기 불편	상세/e-Ticket, 진행 재개 추가
bus link	도시만 선택 가능	운행/좌석 선택 추가

의도적 한계

후속 과제로 명확히 둔다.

- 1 Admin 인증은 backend stub 수준
- 2 현재 예약 flow는 성인 1명/economy 중심
- 3 e-Ticket PDF 다운로드는 renderer abstraction만 준비
- 4 실제 Skypass HTTP client는 Adapter 교체 지점으로 남김

결론: 기능 확장 축을 패턴으로 분리했다

최종 발표 마지막 문장

- 1 기능이 늘어날수록 if문으로 버티지 않고, 변하는 부분을 객체 역할로 분리했다.
- 2 JavaFX UI는 그 패턴들을 실제로 시연 가능하게 만든 Boundary다.
- 3 따라서 화면, 터미널 로그, VS Code 코드가 같은 설계 구조를 가리킨다.